Markov Decision Processes for Robot Battery Management

1. Problem Explanation

This assignment models a decision-making system for an autonomous robot managing its battery life while executing tasks. The environment is framed as a Markov Decision Process (MDP) characterized by states, actions, transition probabilities, and rewards.

- **States:** The robot's battery can be High, Low, or Charging.
- **Actions:** The robot can either Search (execute a task for points) or Wait (conserve energy).
- **Dynamics:** Actions have stochastic (randomized) outcomes. For example, searching with a High battery usually keeps the battery High, but sometimes drains it to Low. Searching on a Low battery might yield a high reward, but carries a risk of completely draining the battery, forcing a transition to the High state accompanied by a severe recharge penalty (-3).

The goal of this assignment is to evaluate a baseline set of rules (a policy) and then use algorithmic methods to find the absolute mathematically optimal ruleset to maximize the robot's long-term point accumulation.

Task 1: Iterative Policy Evaluation

Objective: To determine the long-term expected value of starting in each battery state if the robot strictly follows a fixed baseline policy: [High: Search, Low: Wait, Charging: Wait].

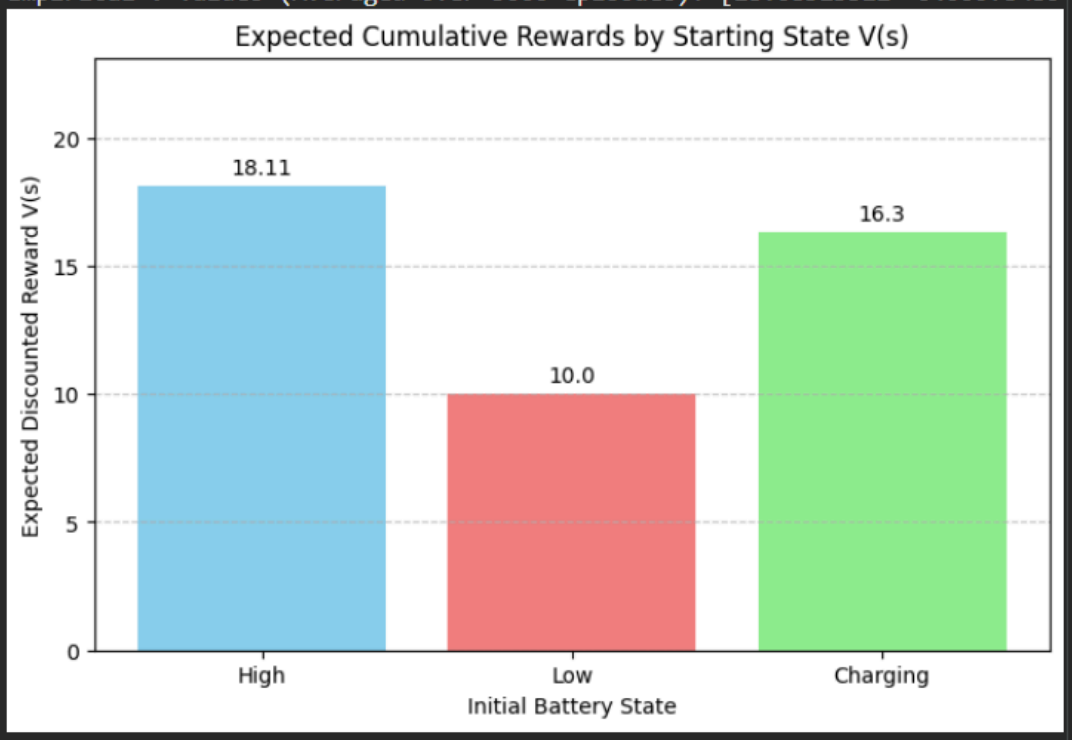
Theoretical Approach: We used the Bellman Expectation Equation. This algorithm initializes the value of all states to zero and iteratively updates them. In each sweep, the value of a state is updated to be the immediate reward of the fixed action, plus the discounted expected value of the next possible states. The algorithm stops when the values stop changing significantly (convergence).

Results & Output Interpretation: Based on the generated bar chart and terminal output, the evaluation converged in 133 iterations.

- Starting in **High** yielded an expected cumulative reward of 18.11.
- Starting in **Low** yielded 10.0.
- Starting in **Charging** yielded 16.30. The chart visually confirms that under this specific conservative policy, starting with a High battery provides the best long-term payoff, while starting Low severely limits earning potential because the policy forces the robot to safely "Wait" and earn minimal points.

```
--- Task 1.2: Policy Evaluation ---
Number of iterations taken to converge: 133
Converged V values: [18.1080999   9.99999179 16.29728909]

--- Bonus: Gymnasium Empirical Evaluation ---
Empirical V values (Averaged over 5000 episodes): [18.08513312  9.99973439
```



Task 2: Value Iteration

Objective: To find the optimal value function and extract the best possible policy without being tied to a fixed initial rule.

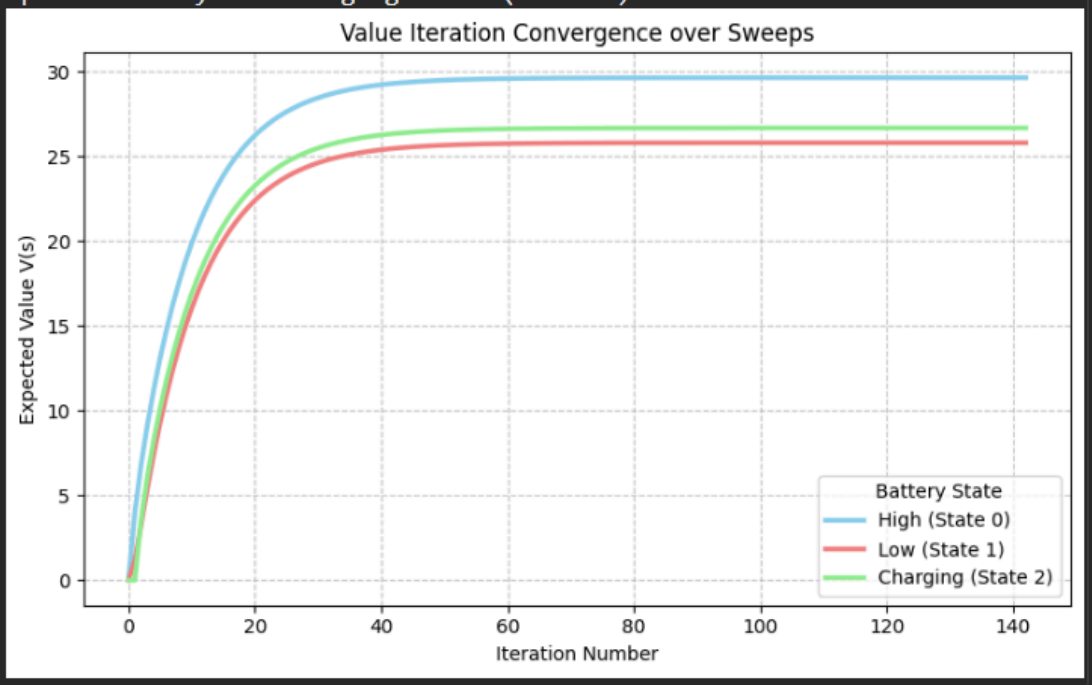
Theoretical Approach: Value Iteration uses the Bellman Optimality Equation. Instead of following a set rule, during every single sweep, the algorithm looks at all possible actions from a state and greedily chooses the one that offers the maximum combined immediate and future reward. Once these optimal values stabilize, we extract the policy by doing one final check to see which action produces that maximum value.

Results & Output Interpretation: The output plot shows a smooth, asymptotic convergence curve for all three states. The algorithm took 142 iterations to find the absolute maximum expected rewards: approximately 29.64 (High), 25.81 (Low), and

26.68 (Charging). Extracting the policy from these optimal values revealed the true best strategy: **[High: Search, Low: Search, Charging: Wait]**

```
--- Task 2.1: Value Iteration ---
Converged in 142 iterations.
Optimal V* values: [29.64382671 25.80821027 26.67944315]

--- Task 2.2: Extract Optimal Policy ---
Optimal Policy for High: Search (Index 0)
Optimal Policy for Low: Search (Index 0)
Optimal Policy for Charging: Wait (Index 1)
```

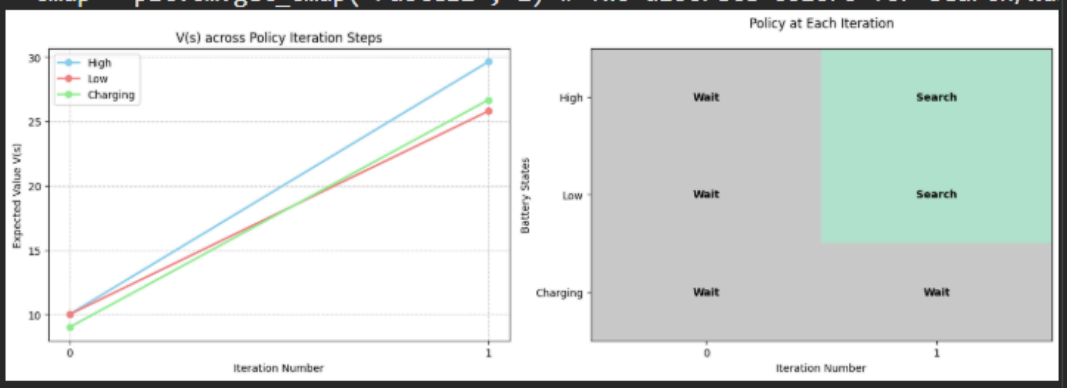


Task 3: Policy Iteration

Objective: To find the optimal policy using a different algorithmic approach that alternates between evaluating a policy and improving it.

Theoretical Approach: Policy Iteration starts with an arbitrary policy (in this case, always Wait). It fully evaluates this policy until the values converge (like in Task 1). Then, it acts greedily upon those converged values to formulate a better policy. This two-step cycle repeats until the policy itself stops changing.

Results & Output Interpretation: The provided heatmap and line plots show a dramatic shift. The algorithm started with an all-Wait policy at Iteration 0. After fully evaluating it, the improvement step instantly recognized that Searching is vastly superior. By Iteration 1, the policy updated to **[High: Search, Low: Search,**

Charging: Wait] and stabilized immediately. The final values match the Value Iteration results exactly (~29.64, 25.81, 26.68), confirming both methods found the mathematical truth.

```
--- Final Results ---
Optimal Policy pi*: [0 0 1]
Optimal Values V*: [29.64 25.81 26.68]
/tmp/ipykernel_9783/654786771.py:125: MatplotlibDeprecationWarning: The ge
  cmap = plt.cm.get_cmap('Pastel2', 2) # Two discrete colors for Search/Wa
```